EXPERIMENT - 10

AIM :

To write a Python program to implement the A* search algorithm to find the least-cost path from a starting node to a goal node using path cost and heuristic estimation..

ALGORITHM:

1. Start.
2. Initialize the starting node and goal node.
3. Set the starting node's path cost to zero.
4. Calculate the evaluation function using \(f(n)=g(n)+h(n)\), where \(g(n)\) is the path cost and \(h(n)\) is the heuristic estimate.
5. Select the node with the lowest evaluation value.
6. If it is the goal node, display the solution path and total cost.
7. Otherwise, expand the node and calculate the costs of its neighboring nodes.
8. Update the costs and parent information of the neighboring nodes.
9. Repeat until the goal is reached or no nodes remain to explore.
10. Stop.

PROGRAM :

untitled (3.14.8)

File Edit Format Run Options Window Help

```
import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('E', 1)],
    'D': [('G', 5)],
    'E': [('G', 3)],
    'G': []
}

h = {'A': 7, 'B': 6, 'C': 4,
     'D': 4, 'E': 2, 'G': 0}

def astar(start, goal):
    q = [(h[start], 0, start, [start])]
    best = {start: 0}

    while q:
        f, g, node, path = heapq.heappop(q)

        if node == goal:
            print("Path:", path)
            print("Cost:", g)
            return

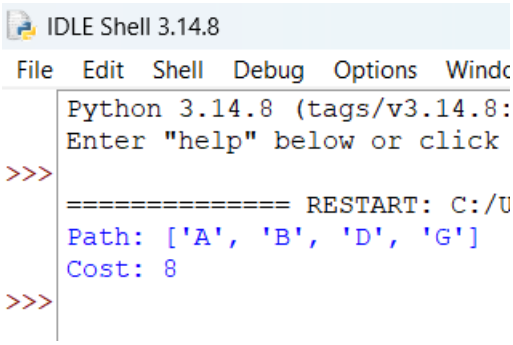
        if g != best[node]:
            continue

        for nxt, w in graph[node]:
            new_g = g + w

            if new_g < best.get(nxt, float('inf')):
                best[nxt] = new_g
                heapq.heappush(
                    q, (new_g + h[nxt], new_g,
                        nxt, path + [nxt])
                )

astar('A', 'G')
```

OUTPUT:



RESULT:

Thus, the Python program to implement the A* algorithm was executed successfully, and the path from the start node to the goal node was found.